UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE
CORSO DI LAUREA IN INFORMATICA
INSEGNAMENTO DI BASE DI DATI E SISTEMI INFORMATIVI I

Progetto Laboratorio Sistemi Operativi

Gennaro De Luca (N86004471), Francesco Simone (N86004627), Mario Flavio Di Blasio (N86004764)

Febbraio 2026

Indice

1 Introduzione **3**

2 Scelte Implementative **3**
2.1 Architettura del Progetto 3
2.2 Tecnologie utilizzate 3

3 Gestione del Server **4**
3.1 La struttura del game 4
3.2 Salvataggio partite 4
3.3 Gestione delle partite 4
3.4 Gestione delle Connessioni 5

4 Gestione Client **6**
4.1 Struttura Client 6
4.2 Comunicazione 6

5 Comunicazione client-server **6**
5.1 Protocollo di comunicazione 6

1 Introduzione

La traccia scelta ci introduce a sviluppare un applicativo client-server multi-client per ospitare partite di Tris.

Di seguito viene riportata la traccia con i requisiti: Un giocatore può creare una o più partite, ma può giocare solo ad una partita alla volta. Il creatore di una partita può accettare o rifiutare la richiesta di partecipazione alla partita da un nuovo giocatore. Gli stati di gioco possono essere terminata, in corso, in attesa, nuova creazione. Gli stati di terminazione di partita possono essere vittoria, sconfitta, pareggio rispetto al giocatore. In base allo stato di gioco di ogni singola partita, tutti i giocatori collegati al server dovranno ricevere un messaggio diverso. Per esempio, "in attesa" tutti i giocatori vengono invitati a partecipare alla partita. Le partite devono essere identificate in maniera univoca. A fine partita (terminata), i giocatori di ogni partita possono scegliere se iniziare o meno un'altra partita.

- Opzionale: Il vincitore di una partita può decidere se fare un'altra partita - in questo caso, se non era il proprietario, diventa il proprietario della partita e attende un nuovo giocatore. Il perdente deve lasciare la partita. Se c'è pareggio, entrambi i giocatori possono decidere se farne un'altra.

2 Scelte Implementative

2.1 Architettura del Progetto

Il progetto è stato concepito con un'architettura **client-server multi-thread** per garantire la gestione simultanea di più partite e l'interazione tra i giocatori. La **logica e la gestione** delle partite avviene tutto lato server.

2.2 Tecnologie utilizzate

Di seguito viene riportata una tabella che descrive tutte le tecnologie utilizzate e le motivazioni.

Categoria	**Tecnologia**	**Motivazione**
Server-Side	C	Scelto per la sua capacità di gestire le risorse di sistema a basso livello. L'uso delle librerie standard **POSIX** (come pthreads) permette una corretta gestione della concorrenza. Utilizzati **mutex** e **condition variables** per gestire rispettivamente l'atomicità sul vettore delle partite e l'attesa di uno dei client durante le fasi di ricerca e accettazione di un utente in una partita.
Client-Side	Python	Selezionato per la rapidità di sviluppo e la semplicità di implementazione di un'interfaccia client leggera e funzionale, facilitando l'interazione dell'utente con il server C.
Networking	TCP/IP	Usato per la sua affidabilità nell'invio dei pacchetti garantendo anche anche l'ordinamento corretto dei messaggi. Inoltre, con il TCP, il server rileva disconnessioni o chiusure tramite il valore 0 di recv. Ciò permette la pulizia immediata della sessione e la notifica all'altro giocatore.
Containerizzazione	Docker	Utilizzato per semplificare la distribuzione e la portabilità del progetto utilizzando due container: uno per il client e uno per il server.
Versionamento	Git	Impiegato come strumento di versioning.

3 Gestione del Server

Il server rappresenta il cuore logico del sistema e ha il compito di gestire l'intera evoluzione delle partite di Tris.

3.1 La struttura del game

Una partita viene definita tramite una apposita struct che contiene tutte le informazioni necessarie per la partita. Fondamentale è l'integrazione di mutex e variabili di condizione direttamente nella struttura: questa scelta permette di gestire in sicurezza l'accesso ai dati da parte di più thread, in modo da gestire la sincronizzazione tra thread ed evitare casi di race condition critiche.

```c
typedef struct Game{
    int id;
    int id_player1;
    int id_player2;
    char table[3][3];
    GameState state;
    int turn;
    pthread_mutex_t game_mutex;
    pthread_cond_t cond_approve;
    int rematch_status_player1;
    int rematch_status_player2;
} Game;
```

Listing 1: struttura game

3.2 Salvataggio partite

Le partite vengono salvate in un array di puntatori a `Game` allocato dinamicamente. In particolare, il size del vettore cresce quando una partita viene creata e diminuisce quando un game viene terminato. Questo meccanismo assicura che il server utilizzi solo le risorse strettamente necessarie al numero di utenti connessi. La logica di creazione e inizializzazione, di inserimento e rimozione di elementi dal vettore è incapsulata nello script `game_vector.c`, in una struttura dati chiamata `game_vector_t`. Determinante, è il mutex `mutex_list` che permette di gestire la sincronizzazione tra i diversi thread, i quali potrebbero tentare di accedere in modo concorrente al vettore. Infatti tutte le operazioni di lettura, scrittura e rimozione acquisiscono il `lock` prima di manipolare il vettore, per poi al termine rilasciarlo con `unlock`, prevenendo in questo modo la corruzione dei dati e le race condition in ambiente multi-thread.

```c
typedef struct game_vector_t{
    Game ** vector;
    pthread_mutex_t mutex_list;
    int current_index;
    int size;
} game_vector_t;
```

Listing 2: struttura game_vector_t

3.3 Gestione delle partite

Le partite vengono gestite tramite il file `game.c` che è il motore del progetto. Al suo interno è implementata la logica del Tris (verifica della vittoria, pareggio o mosse non valide), gestisce anche tutte le operazioni mandate e ricevute dall'utente (creazione di una partita, unione ad una partita, invio di mosse, disconnessione) e aggiorna i giocatori sullo stato della partita. Tutte queste funzionalità vengono gestite da un nucleo centrale, ovvero la funzione game_action, funzione che gestisce tutte le azioni mandate dal client. Questa funzione viene chiamata dal main quando il server crea un nuovo thread (con `pthread_create()`) per gestire il client appenna connesso.

```c
void* game_action(void *arg)
{
    while (1)
    {
        int action = 0;
        //controllo disconnessione utente
        if (recv(sd, &action, sizeof(int), 0) <= 0) {
            break;
```

```
        }
        switch (action)
        {
        case CREATE:
            break;
        case LIST:
            break;
        case JOIN:
        {
            break;
        }
        case MOVE:
        {
            break;
        }
        case REMATCH:
        {
            break;
        }
        case APPROVE:
        {
            break;
        }
        case CANCEL:
        {
            break;
        }
        default:
            break;
        }
    }
    return NULL;
}
```

Listing 3: game_action()

3.4 Gestione delle Connessioni

Il server utilizza un'architettura multi-threading per gestire più client contemporaneamente senza blocchi.

Per ogni connessione accettata, viene creato un thread dedicato che prende in carico l'intero ciclo di vita dell'utente. Per garantire una comunicazione sicura, il socket descriptor del client viene allocato dinamicamente nella memoria heap (in particolare nel campo `id_player1/2` della struct `Game`), in modo che tutti i thread possano accedere ad esempio al socket descriptor del giocatore avversario.

Nel momento in cui il client si è disconnesso, si provvederà alla chiusura del socket con la funzione `close(sd)`. Successivamente, tramite la funzione `pthread_detach()`, il thread viene sganciato dal processo principale: questo assicura che le risorse vengano liberate automaticamente alla disconnessione dell'utente, prevenendo memory leak.

```
    int port = 5200;
    char *env_port = getenv("PORT");
    if (env_port != NULL) {
        port = atoi(env_port);
    }

    int sd;
    sd = start_server(port, 5);
    init_game_session();

    while(1) {
        int newsd = accept_client(sd);
        handle_client(newsd, game_action);
    }

    close_game_session();
    close_server(sd);

    return 0;
```

Listing 4: Inizializzazione Server

```c
int handle_client(int newsd, void *(*handler)(void *)) {

    int* new_sd_ptr;
    pthread_t tid;

    new_sd_ptr = malloc(sizeof(int));
    *new_sd_ptr = newsd;
    if (pthread_create(&tid, NULL, handler, (void *)new_sd_ptr) != 0) {
        perror("Errore creazione thread");
        close(newsd);
        free(new_sd_ptr);
        return -1;
    }

    pthread_detach(tid);
    return 0;
}
```

Listing 5: Handle Client

4 Gestione Client

L'interfaccia utente ‘e stata sviluppata in Python utilizzando la libreria grafica Tkinter. Questa scelta ‘e stata dettata dalla necessit‘a di creare un client leggero, funzionale e rapido da implementare, capace di gestire agevolmente sia la visualizzazione della griglia di gioco che la comunicazione asincrona con il server in C.

4.1 Struttura Client

Il client non contiene nessuna logica per quanto riguarda la gestione delle partite, si limita solo ad aggiornare l'interfaccia grafica. Di fatto, la struttura principale del client è costituita da funzioni di callback che, alla pressione di un bottone, inviano al server l'apposito codice con i dati necessari ad interpretarlo eventualmente aspettando una risposta dal server e reagendo di conseguenza (ad esempio il bottone "Connetti" invia al server un codice apposito seguito dall'ID della partita alla quale ci si sta connettendo e attende una conferma, se riceve un codice di accettazione viene mostrata la griglia di gioco mentre se riceve un codice di declino viene mostrato un errore).

4.2 Comunicazione

Il client gestisce il ricevimento dei dati inviati dal server tramite un thread dedicato in modo da non bloccare l'interfaccia. Inoltre, i dati rappresentanti numeri interi vengono codificati in "Little-Endian" e i dati rappresentanti stringhe vengono codificati in "UTF-8" per permettere ai linguaggi C e Python di scambiarsi i dati correttamente senza errori di interpretazione.

5 Comunicazione client-server

Come detto in precedenza, il socket è basato sul protocollo TPC/IP per una corretta e affidabile gestione di pacchetti mandati tra client e server, nonché per la verifica della disconnesione dell'utente.

5.1 Protocollo di comunicazione

Abbiamo definito un protocollo tra client-server per standardizzare e rendere robusta la comunicazione tramite numeri interi. Di seguito viene riportata una tabella con la descrizione degli intervalli di codice utilizzati.

Intervallo	**Significato**	**Esempio**
200-299	Codici per gestire l'accesso ad una partita (richiesta, accettazione, rifiuto, partita piena)	`MSG_JOIN_REQUEST` = 201: codice che notifica il creatore che un player vuole entrare in partita
300-399	Codici relativi all'avvio effettivo del gioco e alla gestione dei ruoli	`MSG_CHANGE_OWNER` = 303: notifica allo sfidante che è diventato il nuovo proprietario della lobby
400-499	Codici dedicati alla fase di negoziazione della rivincita (Rematch)	`MSG_REMATCH_REQUEST` = 400: inviato dal server per chiedere ai giocatori se intendono rigiocare
500+	Codici di errore per segnalare fallimenti critici o risorse non trovate	`ERR_JOIN_OWNER_LEFT` = 501: segnala allo sfidante che il proprietario si è disconnesso